(No Waste)

Software Design Document

Names:

Maryam Noureldin

Mennatallah Zaid

Marina Mikhael

Eman Abd Elhady

Farah Habib

Date: 10/14/2026

Table of Contents

1. Introduction

1.1 Purpose
1.2 Scope
1.3 Overview
1.4 Reference Material
1.5 Definitions and Acronyms

2. System Overview

3. System Architecture

3.1 Architectural Design
3.2 Decomposition Description
3.3 Design Rationale

4. Data Design

4.1 Data Description
4.2 Data Dictionary

5. Component Design

6. Human Interface Design

6.1 Overview of User Interface
6.2 Screen Images
6.3 Screen Objects and Actions

7. Requirements Matrix

8. Appendices

1. Introduction

1.1 Purpose

This Software Design Document (SDD) describes how NoWaste, a location-based mobile app that reduces food waste in Egypt, is built to satisfy the NoWaste Software Requirements Specification (SRS) version 1.0. It translates the SRS requirements into components, interfaces and data. Its readers are the team's developers and testers, and the course instructor and teaching assistants. This version is the preliminary design: the overall system architecture and data architecture. Detailed design follows in later sprints, and the document is updated every sprint.

1.2 Scope

The design covers:

- the mobile app used by consumers, vendors, charity and composting partners and administrators;
- a backend that serves them through a REST API, with a separate worker process for timed rules;
- a PostgreSQL database with the PostGIS extension for location queries, and object storage for photos;
- the external services the system calls: AI services (photo recognition, language model, speech), a payment gateway, a map service, a push notification service, and an email and SMS service.

It does not cover delivery of food, and nothing in the design certifies food as safe to eat. The design follows the SRS: every component traces to SRS functional requirements (Section 7), and the quality attributes in SRS Section 5 drive the architecture (Section 2.3).

1.3 Overview

This document follows a summary of IEEE Std 1016-2009.

- Section 2 gives the system overview, context and main design drivers.
- Section 3 gives the architecture: layers, five UML component diagrams, the component catalogue, interfaces, main interaction flows and the design rationale with patterns and alternatives.
- Section 4 describes the data and gives a data dictionary.
- Section 5 gives pseudocode for the components with the most important business rules.
- Section 6 describes the user interface.
- Section 7 traces each component to the SRS functional requirements and user stories.
- Section 8 lists technical risks and the appendices.

1.4 Reference Material

- Institute of Electrical and Electronics Engineers. (2009). IEEE standard for information technology—Systems design—Software design descriptions (IEEE Std 1016-2009).
- NoWaste Software Requirements Specification (SRS), version 1.0. Requirement IDs in this document (FR, E, NFR, BR, IF, TBD, US) refer to the SRS.

1.5 Definitions and Acronyms

Domain terms (listing, Discounted Surplus Bag, Donation, Compost offer, Quick Post, hold, waitlist, pickup code, perishability category, no-show, Zakat and others) are defined in SRS Appendix A and are used here with the same meaning. Terms used only in this document:

Term	Meaning
Adapter	A component that wraps one external service behind an interface, so the service can fail or be replaced without changing business logic
API	Application programming interface
Component	A replaceable part of the system with provided and required interfaces (C-nn)
Interface (Ixxx)	A named set of operations a component provides (ball) or requires (socket) in the UML component diagrams
JWT	JSON Web Token, the signed access token sent with every request
PostGIS	PostgreSQL extension for storing locations and calculating distances
RBAC	Role-based access control: what a user may do depends on their role
REST	Style of web API where the app requests resources over HTTPS
RTL	Right-to-left layout, used for Arabic
UML	Unified Modeling Language
Worker	The background process that runs time-based rules (C-19)

2. System Overview

NoWaste is a client-server system: one React Native app for all roles, and a modular-monolith backend with a separate worker. Vendors post surplus food, AI drafts the listing, and a routing recommender suggests the destination: discounted sale to nearby consumers, donation to charity, composting, or landfill as a last resort. The vendor confirms every AI result and every decision.

2.1 Architectural style

One cross-platform React Native app serves five roles (consumer, vendor, charity partner, composting partner, administrator) and shows each role its own screens. The app talks over HTTPS/REST to a single backend that is split into layers (API, application services, domain, data access) and into modules that match the six SRS features (4.1 to 4.6). The backend reaches each external service only through an adapter, so any of them can fail or be replaced without touching business logic. A separate worker process runs the time-based rules (holds, waitlist offers, automatic charity offers, uncollected Zakat refunds). Data lives in PostgreSQL with PostGIS; photos live in object storage.

2.2 Context

External party	Interaction	SRS
Consumer, vendor, charity partner, composting partner, administrator	Use the mobile app; what they see depends on their role	2.3, 3.1
AI vision service	Classifies the food photo: food type, food category, estimated quantity, calorie range and a confidence value; drafts the listing in Arabic and English	3.3, FR-LST-03, FR-LST-07
Language model service	Interprets typed or spoken (already transcribed) requests for the AI assistant	3.3, FR-AST-01 to FR-AST-03
Speech service	Speech-to-text and text-to-speech in Arabic and English; keeps no audio	3.3, NFR-11b
Payment gateway	Card payments, Zakat payments and refunds; card data never reaches NoWaste	3.3, NFR-10
Map service	Map tiles drawn in the app; listing coordinates come from the backend	3.3, FR-RES-01
Push notification service	Delivers notifications to Android and iOS devices	3.3, FR-RES-11
Email and SMS service	Sends password-reset codes	3.3, FR-ACC-05

Distances and "within 10 km" searches are calculated by PostGIS in the database, not by the map service (SRS 3.3, NFR-01).

2.3 Main design drivers

These SRS requirements shape the architecture most:

- **NFR-03 Reliability:** every AI service is optional. If it fails, vendors post by hand and consumers still search and reserve (E-LST-01, E-AST-02).
- **NFR-05 Food safety:** AI never certifies food. The vendor confirms every AI-filled field, the expiry date caps the pickup window, and missing or uncertain information defaults to Highly perishable (FR-RTE-02).
- **NFR-06, NFR-07 Arabic and accessibility:** right-to-left layout and screen-reader support are built into the client from the first sprint.

- **NFR-08 Low connectivity:** photos compressed to 500 KB or less, small payloads, and a local cache of active reservations and pickup codes (E-RES-06).
- **NFR-09, NFR-10, NFR-11 Security and privacy:** RBAC in one place, salted password hashes, login rate limiting, card data only at the gateway, consumer location and voice recordings not stored.
- **NFR-12 Availability:** when the payment gateway, map, push, or email and SMS service is down, functions that do not need it keep working. Each service sits behind its own adapter for this reason.
- **NFR-13, NFR-14 Maintainability and portability:** automated tests on every merge with mocked adapters; one shared codebase for Android and iOS.
- **Correct reservations:** bags are never over-reserved (FR-RES-07, E-RES-02), and the timers in SRS 5.5 (45-minute hold, 20-minute check-in, 10-minute waitlist offer, 30-minute waitlist cut-off) fire reliably, even after a restart.

2.4 Deployment view

The mobile app (Android and iOS) calls the backend API over HTTPS with TLS 1.2 or later (IF-COM-01). The backend API and the worker run as two processes from the same codebase. Both are restarted automatically on failure by the hosting platform, and both use the same PostgreSQL database and object storage. GitHub Actions runs the automated tests on every merge (NFR-13). During the course the payment gateway runs in sandbox mode only (IF-SW-07). The backend framework and service providers are still open (SRS TBD-2).

3. System Architecture

NoWaste is decomposed into 30 components (C-01 to C-30): the app, the API layer, 16 application services, the worker, 8 adapters, data access and two data stores. They are arranged in layers, and each layer calls only the layer below it.

3.1 Architectural Design

3.1.1 Layers

Layer	Components	Responsibility
Client	C-01 Mobile App, C-24 Map View	Screens for each role, local validation, Arabic and English with RTL, offline cache of active reservations, photo compression, speech capture and playback, map drawing
API	C-02 API Layer	REST endpoints, authentication, RBAC, input validation, login rate limiting
Application services	C-03 to C-18	One service per feature area; implements the SRS business rules and use cases
Domain	(inside the services)	Entities and rules shared by services: listing, reservation, hold, perishability category

Layer	Components	Responsibility
Worker	C-19 Scheduler / Worker	Time-based jobs that call the same application services
Integration adapters	C-20 to C-23, C-25, C-26, C-30	One adapter per external service behind an interface
Data access	C-27 Data Access	Repositories over PostgreSQL; transactions and row locking; no business rules
Data stores	C-28 PostgreSQL + PostGIS, C-29 Object Storage	Relational data and locations; photo files

Rule: a layer calls only the layer below it. Services never call an external service directly; they call adapter interfaces.

3.1.2 UML component diagrams

The component diagram is split into five figures so each stays readable. Figure 1 shows the subsystems. Figures 2 to 4 show the backend services grouped by SRS feature. Figure 5 shows the adapters, external services and data stores. A ball is a provided interface and a socket is a required interface; a ball inside a socket means the component on the socket side depends on that interface. A white box marked "see Figure n" is drawn in full in another figure. Every service also requires IRepository (C-27); to keep the figures readable this is drawn only in Figures 1 and 5.

Figure 1. System overview (subsystems)

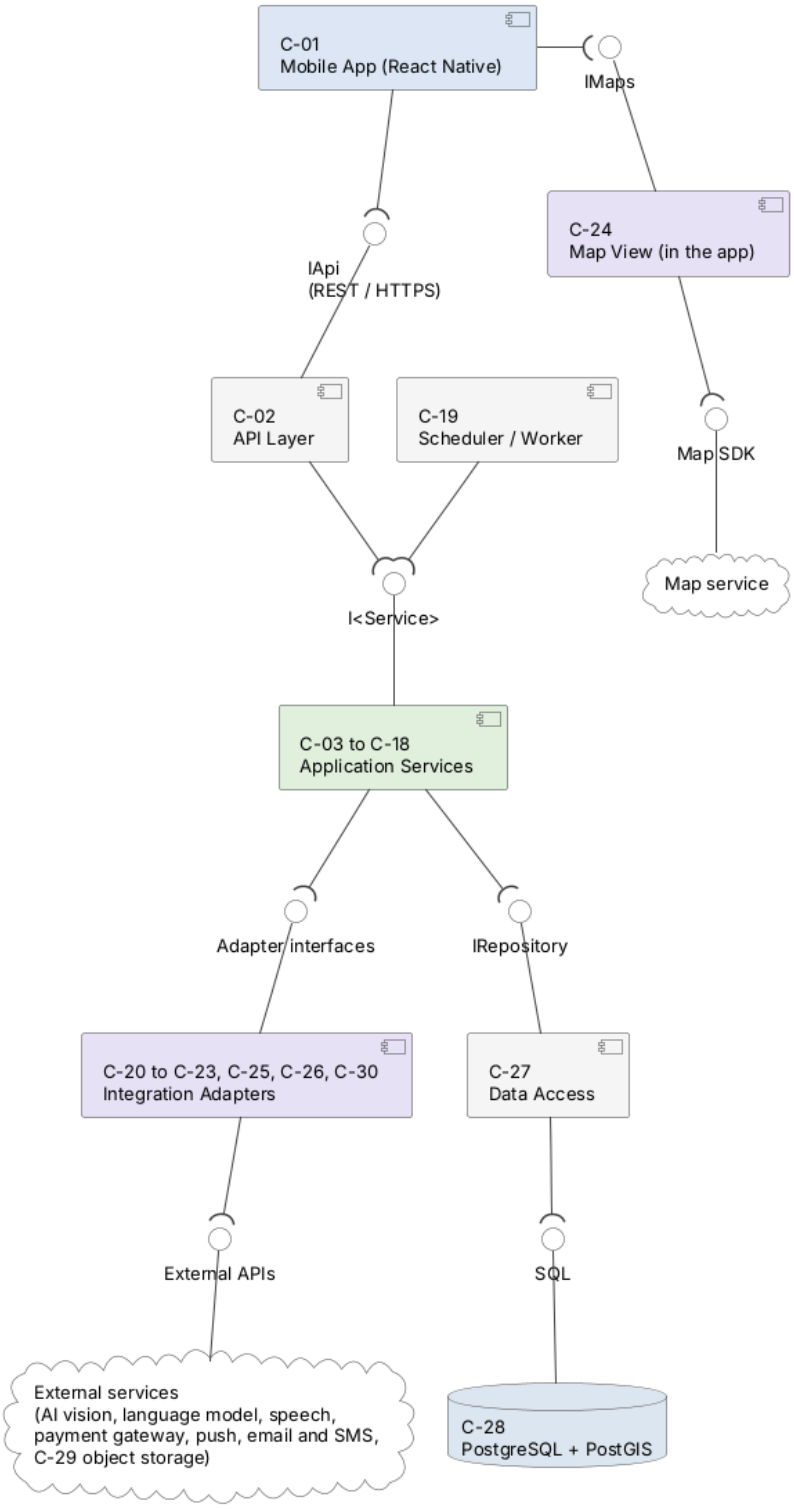


Figure 2. Accounts, listings, perishability and routing (SRS 4.1, 4.2, 4.3)

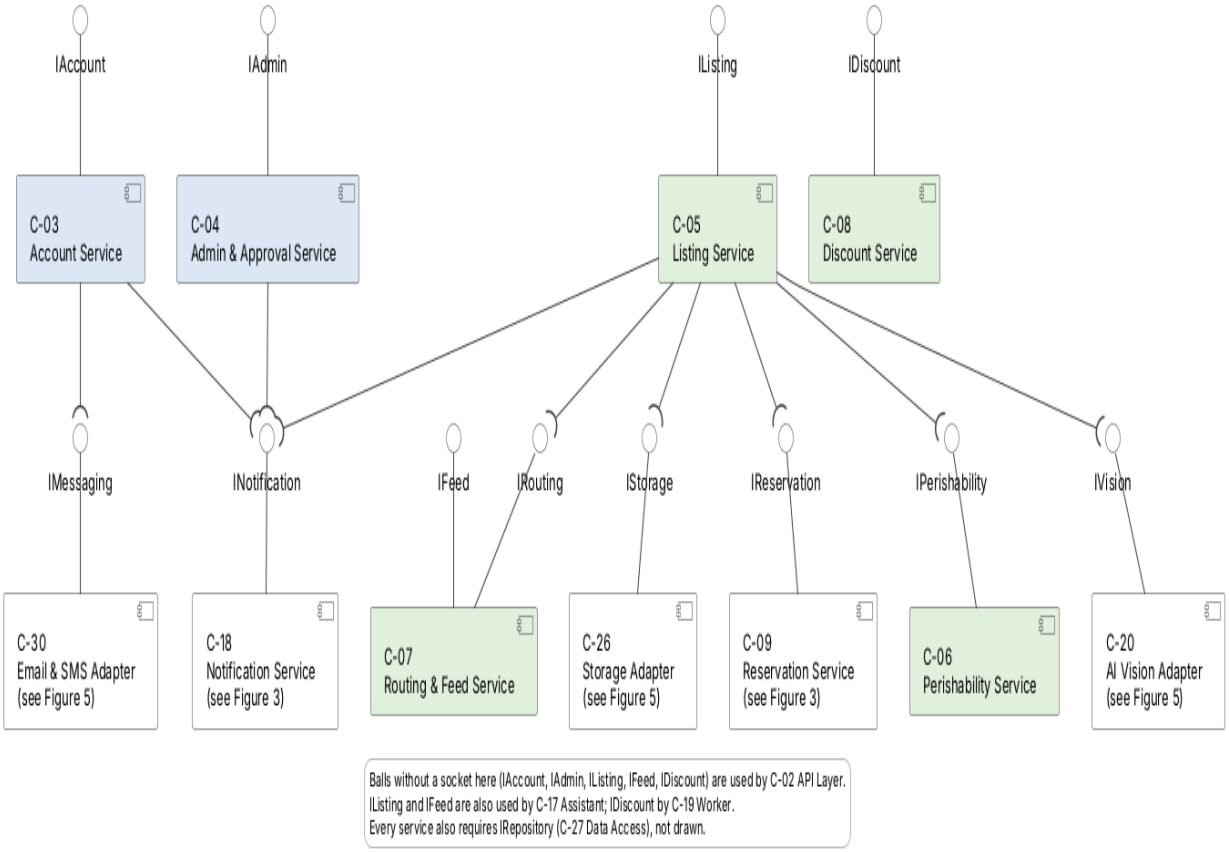
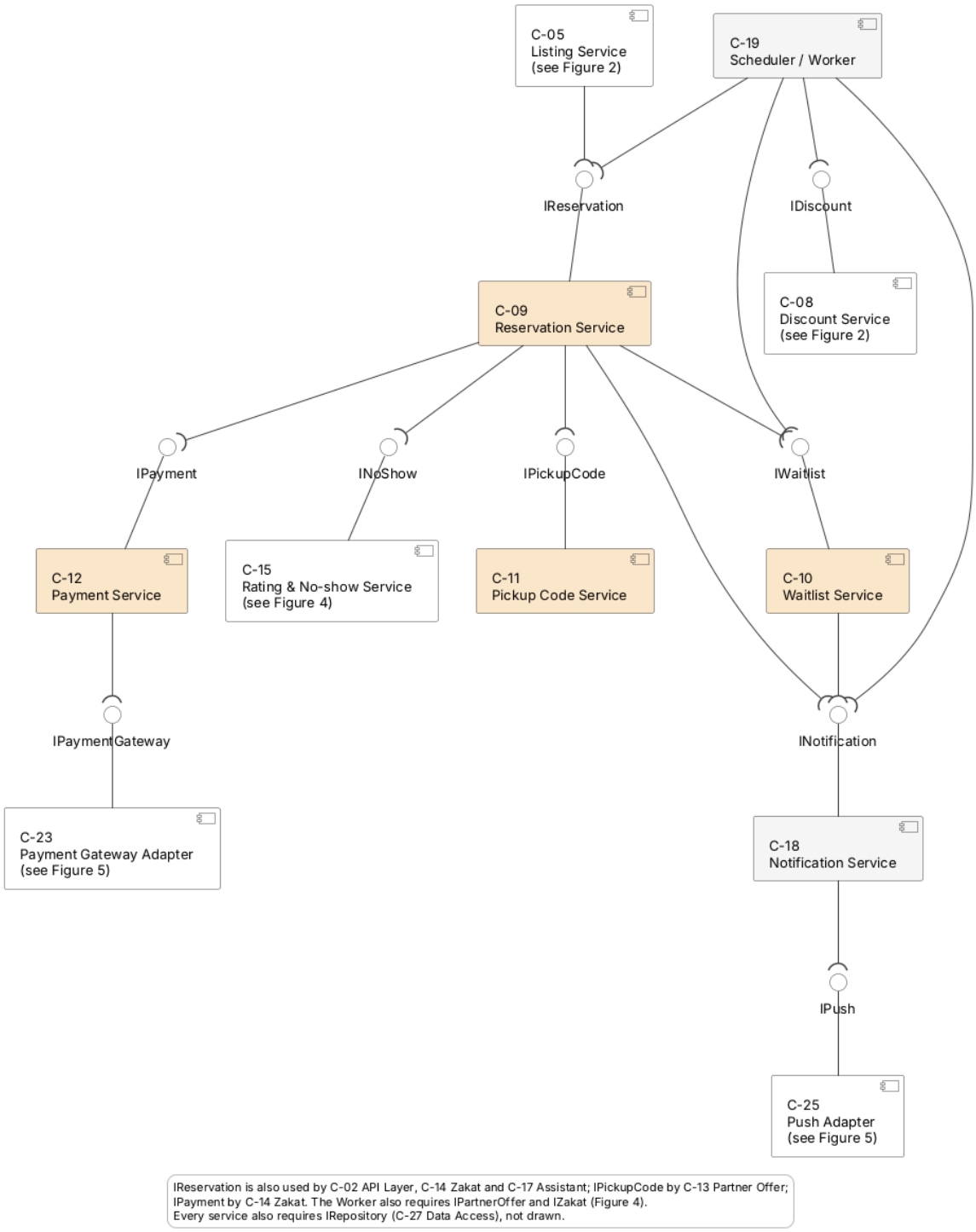
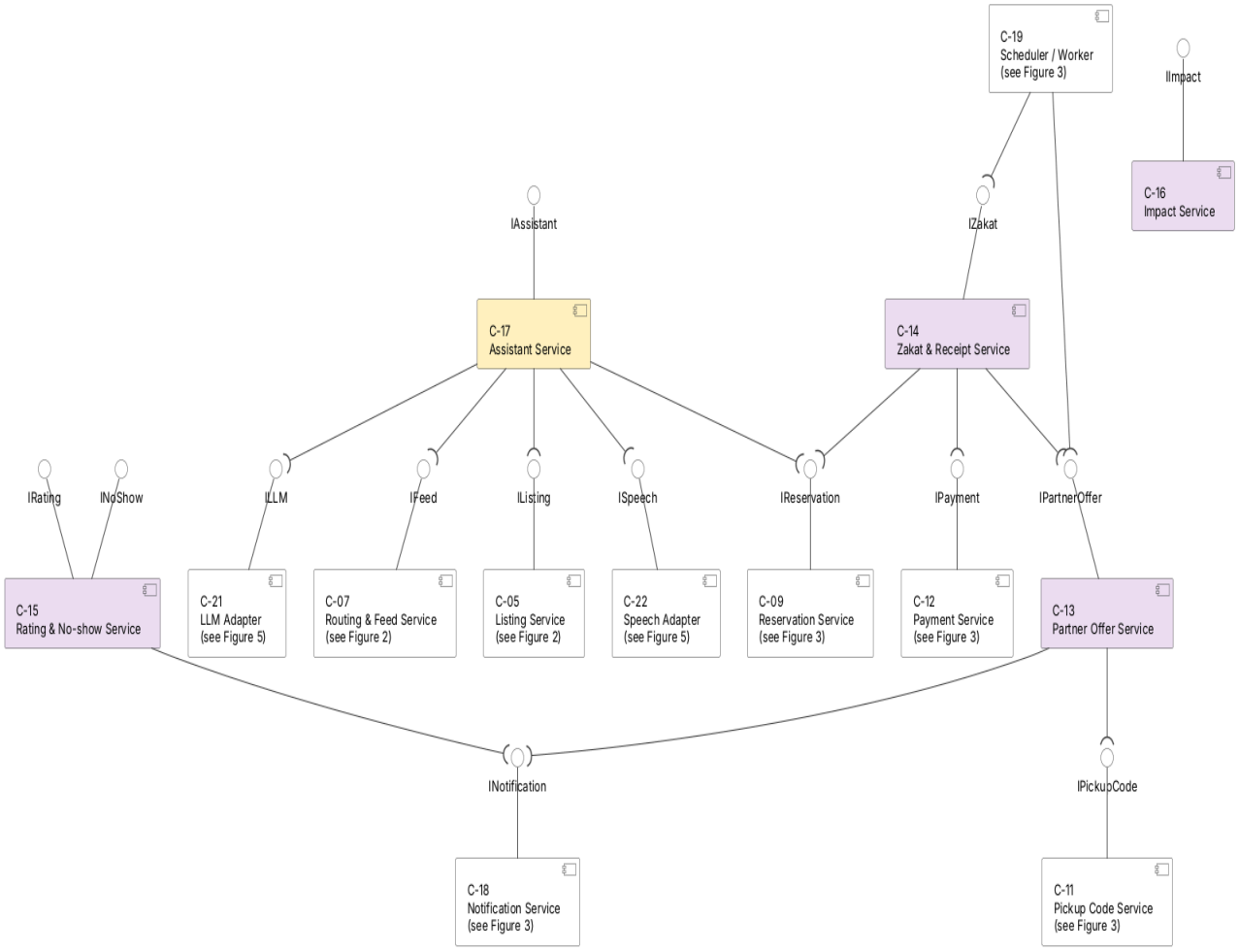


Figure 3. Reservation, payment, waitlist, pickup and notification (SRS 4.4)



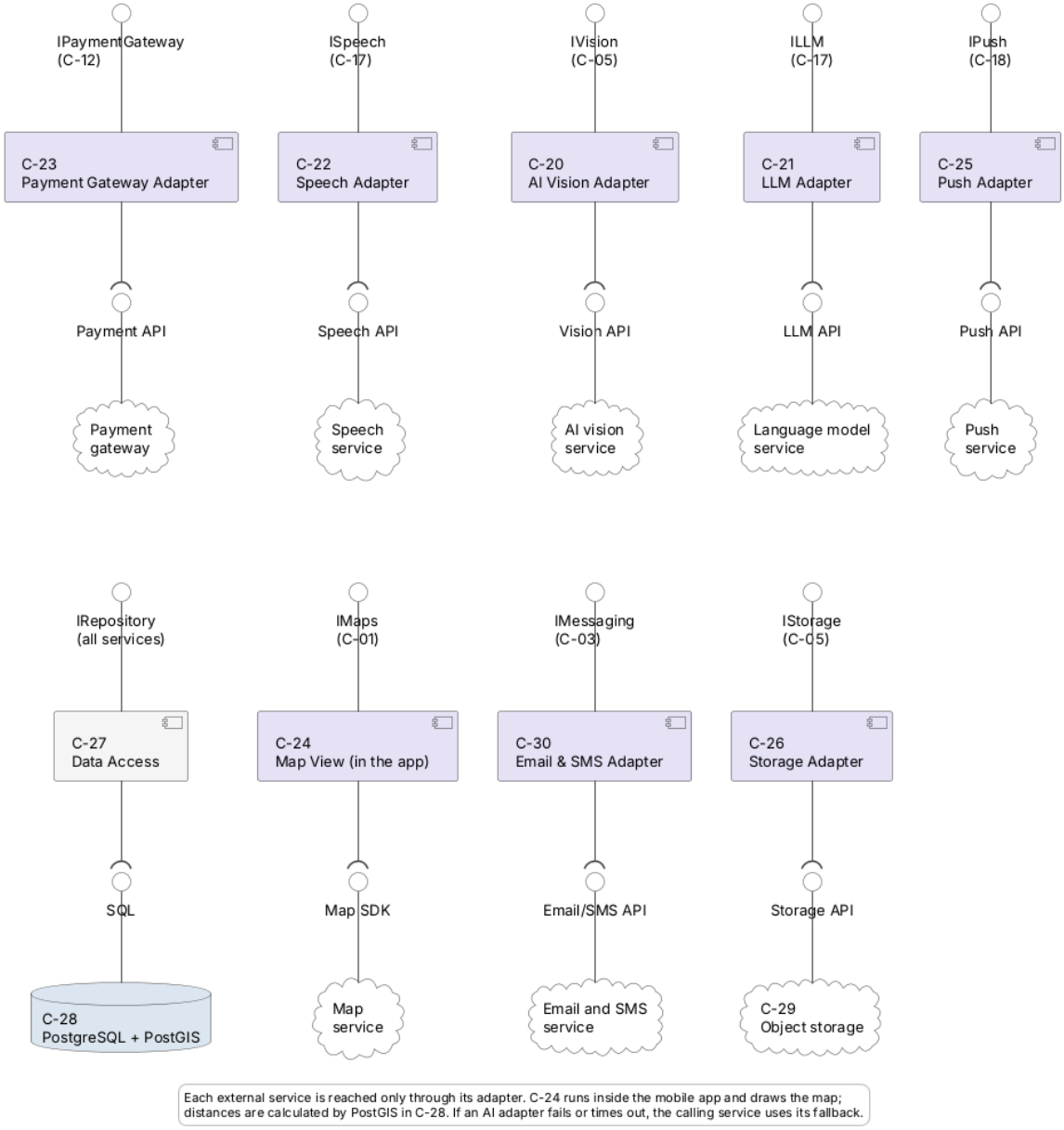
IReservation is also used by C-02 API Layer, C-14 Zakat and C-17 Assistant; IPickupCode by C-13 Partner Offer; IPayment by C-14 Zakat. The Worker also requires IPartnerOffer and IZakat (Figure 4).
Every service also requires IRepository (C-27 Data Access), not drawn.

Figure 4. Partners, Zakat, trust, impact and the AI assistant (SRS 4.3, 4.5, 4.6)



Balls without a socket here (IAssistant, IImpact, IRating) are used by C-02 API Layer; INoShow is used by C-09 Reservation (Figure 3). Every service also requires IRepository (C-27 Data Access), not drawn.

Figure 5. Integration adapters, external services and data



3.2 Decomposition Description

3.2.1 Component catalogue

ID	Component	Responsibility	Provides	Requires	SRS requirements
C-01	Mobile App (React Native)	All screens for the five roles; input validation; Arabic and English with RTL; photo compression	Screens	IApi, IMaps	3.1, IF-UI-01 to 08, NFR-06, NFR-07, NFR-08

ID	Component	Responsibility	Provides	Requires	SRS requirements
		to 500 KB; cache of active reservations and pickup codes; speech capture and playback			
C-02	API Layer	REST endpoints over HTTPS; JWT sessions ended at logout; RBAC on every request; blocks Pending, Rejected and Suspended accounts; locks an account for 15 minutes after 5 failed logins	IApi	All service interfaces	FR-ACC-04, E-ACC-03, E-ACC-05, E-ACC-06, E-LST-06, IF-COM-01 to 04, NFR-09
C-03	Account Service	Consumer registration by email or phone; vendor and partner applications (shop or organisation details, location, accepted food types); login and logout; password reset by code; profile; language	IAccount	INotification, IMessaging	FR-ACC-01 to 06, FR-ACC-09, E-ACC-01, 02, 04
C-04	Admin & Approval Service	Approve or reject applications; suspend or remove vendors and partners (revoke access, hide their listings); review accounts flagged for no-shows	IAdmin	INotification	FR-ACC-07, FR-ACC-08, FR-TGI-04 (review)
C-05	Listing Service	Create listings and Quick Posts; coordinate the AI draft and calorie estimate; check the pickup window against the expiry; publish only when every AI-filled field is confirmed; edit, cancel (cancelling reservations with full	IListing	IVision, IStorage, IPerishability, IRouting, IReservation, INotification	FR-LST-01 to 08, E-LST-01 to 06, NFR-05b, NFR-05c

ID	Component	Responsibility	Provides	Requires	SRS requirements
		refunds) and view listings			
C-06	Perishability Service	Rule-based perishability category and pickup urgency; Highly perishable when information is missing, Unknown or uncertain; only a more perishable category may replace the estimate	IPerishability	none (pure rules)	FR-RTE-01 to 03, E-RTE-01, NFR-05d
C-07	Routing & Feed Service	Ranks destinations (consumers, charity, composting, landfill) by distance, time left, food category and partners' accepted food types and returns the top one; sets the listing type; unfit food to composting only; consumer feed with Highly perishable listings nearby first, then by distance and time left; map filters; Ramadan mode highlighting	IRouting, IFeed	none (PostGIS queries through IRepository)	FR-RTE-04 to 09, E-RTE-02, FR-RES-01, FR-TGI-08
C-08	Discount Service	Rule-based discount suggestion that rises as the pickup deadline nears while bags are unreserved; the vendor chooses whether to apply it	IDiscount	none	FR-LST-08
C-09	Reservation Service	One bag per reservation; card reservation confirmed only after payment	IReservation	IPayment, IPickupCode, IWaitlist,	FR-RES-02 to 09, FR-RES-12, E-RES-01 to

ID	Component	Responsibility	Provides	Requires	SRS requirements
		approval and held for the whole window; no-card hold of 45 minutes from the later of reservation time or window start, capped at window end; one 20-minute check-in; cancellation and refund decision; release of bags; no over-reservation		INoShow, INotification	05, BR-02 to 06
C-10	Waitlist Service	Queue per fully reserved listing; offer a released bag only if 30 or more minutes remain; 10 minutes to accept; then the next person or back to the vendor	IWaitlist	INotification	FR-RES-05, FR-RES-06, BR-04, BR-05
C-11	Pickup Code Service	Issue and verify 6-digit codes for consumer reservations, Donation and Compost pickups; mark the handover Completed	IPickupCode	none	FR-RES-10, E-RES-03, BR-09
C-12	Payment Service	Start and confirm card payments and refunds through the gateway adapter; never stores card data	IPayment	IPaymentGateway	FR-RES-02, FR-RES-12, FR-TGI-05, E-LST-05, E-RES-01, E-TGI-02, NFR-10
C-13	Partner Offer Service	Offer Donations and Compost offers to partners that accept the food type; accept and schedule a pickup within the window; first	IPartnerOffer	IPickupCode, INotification	FR-RTE-10 to 13, E-RTE-03, E-RTE-04

ID	Component	Responsibility	Provides	Requires	SRS requirements
		acceptance wins; partner cancellation and re-offer; automatic charity offers; landfill records			
C-14	Zakat & Receipt Service	Card-paid bag donated as Zakat to a chosen charity that accepts the food type; the bag becomes a Donation; traceable receipt; full refund if not collected	IZakat	IReservation, IPayment, IPartnerOffer	FR-TGI-05, FR-TGI-06, E-TGI-02, E-TGI-03, BR-11
C-15	Rating & No-show Service	Ratings and reviews once per Completed reservation; vendor no-show flags after the window; no-show counting; admin flag at 3 in 30 days; 14-day no-card block	IRating, INoShow	INotification	FR-TGI-01 to 04, FR-RES-13, E-TGI-01, BR-07, BR-08
C-16	Impact Service	Meals rescued, money saved (original value minus price paid), estimated CO2 avoided, bags composted and bags sent to landfill; platform-wide and per user	IImpact	none	FR-TGI-07
C-17	Assistant Service	Interprets typed or spoken requests in Arabic or English; returns the same results as the map search; answers questions, read aloud on request; repeats the reservation and acts only after confirmation; card payment goes to the	IAssistant	ISpeech, ILLM, IFeed, IListing, IReservation	FR-AST-01 to 04, E-AST-01, E-AST-02

ID	Component	Responsibility	Provides	Requires	SRS requirements
		normal payment screen; vendor help			
C-18	Notification Service	Builds and sends push notifications: reservation confirmed, released, waitlist offer, pickup window open, application decision, listing changed or cancelled, account suspended, flagged account for the admin, partner pickup scheduled	INotification	IPush	FR-RES-11, FR-ACC-07, FR-ACC-08, E-LST-05, IF-SW-06
C-19	Scheduler / Worker	Expires holds and records no-shows; runs waitlist offer timers; raises the discount suggestion; sends the window-open notification; makes automatic charity offers (TBD-5 minutes before window end); asks the vendor to record landfill at window end; refunds uncollected Zakat donations	none (calls services)	IReservation, IWaitlist, IDiscount, INotification, IPartnerOffer, IZakat	FR-RES-05, FR-RES-11, FR-RES-13, FR-LST-08, FR-RTE-11, FR-RTE-13, E-TGI-03
C-20	AI Vision Adapter	Wraps the AI vision API: food type, category, quantity, calorie range, confidence, draft text; 10-second time limit	IVision	Vision API	FR-LST-03, FR-LST-07, E-LST-01, NFR-02
C-21	LLM Adapter	Wraps the language model API	ILLM	LLM API	FR-AST-01 to 04
C-22	Speech Adapter	Speech-to-text and text-to-speech in Arabic	ISpeech	Speech API	FR-AST-01, FR-AST-02, NFR-11b

ID	Component	Responsibility	Provides	Requires	SRS requirements
		and English; deletes audio after processing			
C-23	Payment Gateway Adapter	Wraps gateway charge and refund calls; sandbox mode during the course	IPaymentGateway	Payment API	NFR-10, IF-SW-02, IF-SW-07
C-24	Map View (in the app)	Wraps the map SDK in the app: tiles and listing pins from coordinates the API returns	IMaps	Map SDK	FR-RES-01, 3.3 Map service
C-25	Push Adapter	Sends to Android and iOS push services	IPush	Push API	FR-RES-11
C-26	Storage Adapter	Stores and fetches compressed photos	IStorage	Storage API	FR-LST-01, E-LST-02
C-27	Data Access	All reads and writes to PostgreSQL; transactions; row locking; PostGIS distance queries	IRepository	SQL	FR-RES-07, E-RES-02, IF-COM-06
C-28	PostgreSQL + PostGIS	Relational data, locations and distance indexes	SQL	none	NFR-01
C-29	Object Storage	Photo files	Storage API	none	FR-LST-01
C-30	Email & SMS Adapter	Sends password-reset codes by email or SMS	IMessaging	Email/SMS API	FR-ACC-05, E-ACC-04

3.2.2 Generalization and lifecycles

- **Users:** User → Consumer, Vendor (Formal, Informal), Partner (Charity, Composting), Administrator. Administrator accounts are created by the team (SRS 2.7).
- **Listings:** Listing → Discounted Surplus Bag, Donation, Compost offer. The type is set by the destination the vendor chooses (FR-RTE-06). Quick Post is a way of creating a listing, not a type: a Quick Post becomes a Discounted Surplus Bag that is always Highly perishable (FR-LST-02).
- **Account states:** Pending → Approved or Rejected (vendors and partners); Approved → Suspended or Removed (FR-ACC-07, FR-ACC-08).

- **Listing states:** Draft → Active → Fully reserved ↔ Active → Closed, or Cancelled by the vendor (FR-LST-05).
- **Reservation states:** Pending payment → Confirmed (card) or Held (no-card) → Completed, Released, Cancelled or No-show.
- **Partner offer states:** Offered → Accepted (pickup scheduled) → Completed; Accepted → Offered again when a partner cancels; Offered → Landfill recorded when no one takes the food.

3.2.3 Key interfaces

Interface	Main operations
IApi (REST)	POST /auth/register, POST /auth/login, POST /listings, GET /listings/nearby, POST /reservations, POST /reservations/{id}/checkin, POST /pickups/verify, POST /zakat, POST /assistant/message
IVision	classify(photo) → {foodType, foodCategory, quantity, calorieRange?, draftTextAr, draftTextEn, confidence}
IPerishability	estimate(foodCategory?, prepTime?, storage?, expiry?, isQuickPost, aiConfident) → {category, urgency}
IRouting	recommend(listing) → ranked destinations with the top one marked; setListingType(listingId, destination)
IFeed	nearby(consumerLocation, filters, ramadanMode) → listings in feed order
IReservation	reserve(listingId, consumerId, method), checkIn(id), cancel(id), releaseExpired()
IWaitlist	join(listingId, consumerId), offerNext(listingId), accept(offerId), expireOffer(offerId)
IPickupCode	issue(reservationOrOfferId), verify(code, vendorId) → Completed or error
IPayment / IPaymentGateway	charge(amount, token) → approved or declined; refund(paymentId, amount)
IPartnerOffer	offer(listingId), accept(offerId, partnerId, pickupTime), cancel(offerId), recordLandfill(listingId)
IZakat	donate(listingId, consumerId, charityId), markDelivered(id), refundUncollected()
INoShow	record(reservationId, source), countLast30Days(consumerId), isBlocked(consumerId)
INotification	send(userId, type, payload)
IMessaging	sendCode(emailOrPhone, code)
IRepository	Repository per entity: get, save, query, lockForUpdate

3.2.4 Main interaction flows

1. **Create a listing:** App → API → Listing Service → Storage Adapter (photo) and Vision Adapter (draft within 10 seconds, else manual entry) → vendor confirms or edits → Perishability Service

→ vendor confirms the category → Routing Service (top destination) → vendor chooses → listing Active.

2. **Card reservation:** App → API → Reservation Service locks the listing row and reserves one bag → Payment Service → gateway → approved: confirm, issue pickup code, notify; declined: release the bag (E-RES-01).
3. **No-card hold expiry:** Worker → Reservation Service releases the bag and records a no-show → Waitlist Service (only if 30 or more minutes remain) → Notification Service → 10-minute offer timer → next person, or the bag returns to the vendor.
4. **Donation pickup:** Charity partner accepts a Donation and proposes a time → Partner Offer Service checks it is within the window → pickup scheduled, vendor notified, pickup code issued → vendor enters the code at handover.
5. **Zakat donation:** Consumer chooses Donate as Zakat and a charity → Zakat Service reserves the bag, takes card payment, turns the bag into a Donation for that charity and issues a receipt → charity collects with the pickup code → receipt updated; if not collected by window end, the Worker refunds in full.
6. **Assistant reservation:** App → Assistant Service → Speech and LLM adapters → search through IFeed → repeats listing, price, window and method → consumer confirms → Reservation Service (same rules as the screens); card payment opens the normal payment screen.

3.3 Design Rationale

3.3.1 Why this structure

- **Modular monolith, not microservices.** A five-person student team, one semester, one database: a single deployable backend with clear modules is simpler to build, test and run. Modules match the SRS features, so they can be split later.
- **Layered architecture.** Separates screens, rules and data. Business rules (hold lengths, waitlist, refunds) live in one layer and are unit-tested without a database or network (NFR-13).
- **One cross-platform client.** React Native gives one codebase for Android and iOS with Arabic RTL support (NFR-06, NFR-14).
- **Shared services for screens and assistant.** The assistant calls the same services as the screens, so it follows the same rules and returns the same results (FR-AST-01, FR-AST-03).
- **Separate worker with timers stored in the database.** Timers keep running when no one calls the API and survive a restart; an API request is the wrong place to wait 10 minutes.
- **Rules where predictability matters.** Perishability, routing, discount suggestion and no-show counting are rule-based. AI is used only for photo classification, calorie estimates, the assistant and speech, and each has a fallback (NFR-03, NFR-05).
- **PostgreSQL with PostGIS.** Reservations, holds and receipts need transactions and row-level locking to prevent over-reservation, and PostGIS answers "within 10 km" searches in the database (NFR-01).

3.3.2 Patterns used

Pattern	Where	Why
Layered architecture	Whole backend	Separation of concerns, testability
MVVM	Mobile App	Screens (view), screen state (view model), API data (model); role-based screens and RTL
Adapter	C-20 to C-26, C-30	Hides each provider's API; swap or mock without touching services
Facade	C-02 API Layer over the services	One entry point for the app, with authentication and RBAC in one place
Repository	C-27 Data Access	Keeps SQL out of services; easy to mock in tests
Strategy	Reservation method (card or no-card); routing ranking; discount formula	Same flow with interchangeable behaviour
State	Account, listing, reservation and partner offer lifecycles (3.2.2)	Makes allowed transitions explicit and testable
Observer (events)	Reservation released → Waitlist and Notification	Decouples services that react to the same event
Fallback (graceful degradation)	Every AI adapter and every external service	AI failure → manual listing, Highly perishable, normal screens; other services → functions that do not need them keep working (NFR-03, NFR-12)

3.3.3 Alternatives considered

Alternative	Why not chosen
Microservices	Too much deployment and testing overhead for this team and timeline
Native Android and iOS apps	Two codebases; conflicts with NFR-14
AI decides perishability end to end	A photo cannot show how long food has been out; rules plus vendor facts are safer and testable (NFR-05)

Alternative	Why not chosen
Distance calculation in the map service	An extra network call per search and a usage cost; PostGIS does it in the query (NFR-01)
Timers inside the API process	Lost on restart; a database-backed worker survives restarts
Storing photos in the database	Slows queries and enlarges backups

4. Data Design

4.1 Data Description

All structured data lives in PostgreSQL. Locations of vendors and listings are PostGIS GEOGRAPHY(Point, 4326) columns with a spatial index, so "listings within 10 km" is one indexed query (NFR-01). Photos live in object storage (C-29); tables keep only the object key. A consumer's device location is sent with each search but never stored (NFR-11a), and no audio is stored (NFR-11b).

The backend is the single source of truth for bag counts, holds, waitlists and reservation status (IF-COM-06). Reserving a bag runs in one transaction that locks the listing row (SELECT … FOR UPDATE), so two consumers can never take the last bag (E-RES-02). Every timer is stored as a timestamp (hold_expires_at, offer_expires_at, pickup_end) measured by the server clock (IF-COM-07), so the worker can resume after a restart.

Main relationships:

- User 1 ⟶ 0..1 VendorProfile or PartnerProfile (role-specific details).
- Vendor 1 ⟶ many Listings; Listing 1 ⟶ many Reservations (at most bags_total active), many WaitlistEntries, many PartnerOffers.
- Reservation 1 ⟶ 0..1 Payment, 1 PickupCode, 0..1 Rating, 0..1 NoShow, 0..1 ZakatDonation.
- PartnerOffer 1 ⟶ 1 PickupCode once accepted.

4.2 Data Dictionary

Entity	Attributes (type)	SRS
User	id (UUID, PK); email (CITEXT, unique, nullable); phone (VARCHAR, unique, nullable; at least one of email or phone); password_hash (TEXT, salted hash); role (consumer, vendor, charity, composting, admin); status (Active, Pending, Approved, Rejected, Suspended, Removed); language (ar, en); no_card_blocked_until (TIMESTAMPTZ, nullable); flagged_for_admin (BOOLEAN); created_at	FR-ACC-01, NFR-09b, FR-RES-13, FR-TGI-04
VendorProfile	user_id (UUID, PK, FK); shop_name; vendor_type (formal, informal); location (GEOGRAPHY); contact; auto_charity_opt_in (BOOLEAN); average_rating (NUMERIC)	FR-ACC-02, FR-RTE-11, FR-TGI-02

Entity	Attributes (type)	SRS
PartnerProfile	user_id (UUID, PK, FK); organisation_name; partner_type (charity, composting); accepted_food_types (one or more of the six food categories); location (GEOGRAPHY)	FR-ACC-03
Listing	id (UUID, PK); vendor_id (FK); listing_type (Discounted Surplus Bag, Donation, Compost offer); status (Draft, Active, Fully reserved, Closed, Cancelled); is_quick_post (BOOLEAN); food_type; food_category (one of six, or Unknown); photo_key; description_ar; description_en; calorie_min, calorie_max (nullable); bags_total, bags_available (INT); original_value (NUMERIC, nullable for Quick Post); price (NUMERIC, EGP); pickup_start, pickup_end (TIMESTAMPTZ); prep_time (nullable, Unknown allowed); storage (room temperature, chilled, frozen, Unknown); expiry_at (nullable); perishability_category; unfit_for_people (BOOLEAN); location (GEOGRAPHY)	FR-LST-01 to 07, FR-RTE-01 to 07
Reservation	id (UUID, PK); listing_id (FK); consumer_id (FK); method (card, no-card); status (Pending payment, Confirmed, Held, Completed, Released, Cancelled, No-show); price_paid; hold_expires_at (nullable); checked_in (BOOLEAN); created_at	FR-RES-02 to 05, FR-RES-12
Payment	id (UUID, PK); reservation_id (FK); gateway_reference; amount (EGP); status (Approved, Declined, Refunded); refunded_at. No card data	FR-RES-02, NFR-10
PickupCode	id (UUID, PK); reservation_id or partner_offer_id (FK); code_hash; status (Active, Used, Void); used_at	FR-RES-10
WaitlistEntry	id (UUID, PK); listing_id (FK); consumer_id (FK); position; status (Waiting, Offered, Accepted, Expired); offer_expires_at	FR-RES-06
PartnerOffer	id (UUID, PK); listing_id (FK); partner_id (FK, nullable until accepted); status (Offered, Accepted, Completed, Cancelled); pickup_time; is_automatic (BOOLEAN)	FR-RTE-10 to 12
LandfillRecord	id (UUID, PK); listing_id (FK); bags (INT); recorded_at	FR-RTE-13
ZakatDonation	id (UUID, PK); reservation_id (FK); charity_id (FK); amount (EGP); receipt_number (unique); status (Paid, Delivered, Refunded); created_at; delivered_at	FR-TGI-05, FR-TGI-06
Rating	id (UUID, PK); reservation_id (FK, unique: one rating per Completed reservation); stars (1 to 5); review (TEXT, nullable); created_at	FR-TGI-01, E-TGI-01

Entity	Attributes (type)	SRS
NoShow	id (UUID, PK); reservation_id (FK, unique: a missed pickup counts once); consumer_id (FK); source (hold expired, cancelled after start, vendor flag); method (card, no-card); created_at	FR-RES-13, FR-TGI-03, FR-TGI-04
Notification	id (UUID, PK); user_id (FK); type; payload (JSON); sent_at; delivered (BOOLEAN)	FR-RES-11
Settings	configuration values: rule times from SRS 5.5, the automatic charity offer minutes (TBD-5), CO2 avoided per bag, iftar and suhoor times by date	BR-01 to BR-12, FR-TGI-07, FR-TGI-08

5. Component Design

The pseudocode below covers the components that carry the most SRS business rules. Times come from the settings table (SRS 5.5) and the server clock.

5.1 C-06 Perishability Service: estimate

```
estimate(foodCategory, prepTime, storage, expiry, isQuickPost, aiConfident):
  # FR-RTE-02, NFR-05d: missing or uncertain information -> most urgent
  if isQuickPost or expiry is null or prepTime == UNKNOWN or storage == UNKNOWN
     or foodCategory == UNKNOWN or not aiConfident:
    return HIGHLY_PERISHABLE, urgency(HIGHLY_PERISHABLE)
  # FR-RTE-01: the applicable limit is the expiry or an earlier suggested limit
  suggestedLimit = prepTime + shelfLife(foodCategory, storage)
  limit = min(expiry, suggestedLimit)
  hours = hoursBetween(now(), limit)
  if hours <= 6:  category = HIGHLY_PERISHABLE
  elif hours <= 48: category = SHORT_LIFE
  else: category = LONG_LIFE
  return category, urgency(category)   # never a statement that food is safe

confirm(listing, chosenCategory):
  # FR-RTE-03: the vendor may only choose a more perishable category
  if moreUrgentOrEqual(chosenCategory, listing.estimatedCategory):
save(chosenCategory)
  else: reject
```

5.2 C-09 Reservation Service: reserve, check in, release, cancel

```
reserve(listingId, consumerId, method):
  BEGIN TRANSACTION
    listing = SELECT ... FROM listings WHERE id = listingId FOR UPDATE   # E-RES-02
    if listing.status != ACTIVE or listing.bags_available == 0: offer waitlist; fail
    if method == NO_CARD and consumer.no_card_blocked_until > now():
      fail with block end date, offer card payment                       # E-RES-04
    listing.bags_available -= 1                                          # FR-RES-07,
one bag
    if listing.bags_available == 0: listing.status = FULLY_RESERVED
```

```
    r = new Reservation(listingId, consumerId, method)
    if method == CARD: r.status = PENDING_PAYMENT
    else:
      start = max(now(), listing.pickup_start)
      r.hold_expires_at = min(start + 45 min, listing.pickup_end)          #
FR-RES-03
      r.status = HELD
  COMMIT
  if method == CARD:
    result = Payment.charge(listing.price, token)
    if result == DECLINED: releaseBag(r); fail                             # E-RES-01
    r.status = CONFIRMED                                                   # held for
the whole window
  PickupCode.issue(r); Notification.send(consumer, CONFIRMED)

checkIn(r):                                                                # FR-RES-04
  if r.status != HELD or r.checked_in or now() > r.hold_expires_at: refuse   #
E-RES-05
  r.hold_expires_at = min(r.hold_expires_at + 20 min, listing.pickup_end);
r.checked_in = true

releaseExpired():   # run by the Worker
  for r in Reservations where status == HELD and hold_expires_at < now():
    r.status = NO_SHOW; NoShow.record(r, HOLD_EXPIRED)                     # FR-RES-13
    releaseBag(r)

releaseBag(r):                                                             # FR-RES-05
  Notification.send(r.consumer, RELEASED)
  if listing.pickup_end - now() >= 30 min: Waitlist.offerNext(listing)
  else: return the bag to the vendor

cancel(r):                                                                 # FR-RES-12
  if r.method == CARD and now() - r.created_at <= 10 min: Payment.refund(r.payment)
  if r.method == NO_CARD and now() >= holdStart(r): NoShow.record(r,
CANCELLED_AFTER_START)
  r.status = CANCELLED; releaseBag(r)
```

5.3 C-10 Waitlist Service: offer the next person

```
offerNext(listing):
  e = first WaitlistEntry where listing and status == WAITING
  if e is null: return the bag to the vendor
  e.status = OFFERED; e.offer_expires_at = now() + 10 min                  # FR-RES-06
  Notification.send(e.consumer, WAITLIST_OFFER)   # the offer also shows in the app
(IF-SW-06)

expireOffer(e):   # run by the Worker
  e.status = EXPIRED; offerNext(listing)

accept(e, method):
  if now() > e.offer_expires_at: refuse
  e.status = ACCEPTED; Reservation.reserve(listing, e.consumer, method)
```

5.4 C-15 Rating & No-show Service: record a no-show

```
record(reservation, source):
  # a missed pickup counts once, whoever records it first (unique reservation_id)
  if NoShow exists for reservation: return
  if source == VENDOR_FLAG and (now() < pickup_end or reservation.status ==
COMPLETED): refuse   # E-TGI-01
  insert NoShow(reservation, consumer, source, reservation.method)
  # FR-TGI-04: every no-show counts toward the administrator flag
  if count(NoShow for consumer in last 30 days) >= 3:
    consumer.flagged_for_admin = true; Notification.send(admins, ACCOUNT_FLAGGED)
  # FR-RES-13: the block counts only no-card no-shows
  if count(no-card NoShow for consumer in last 30 days) >= 3:
    consumer.no_card_blocked_until = now() + 14 days
```

5.5 C-19 Scheduler / Worker: jobs

The worker polls every minute and handles each due item once (idempotent updates keyed on the stored timestamp), so a restart neither skips nor repeats a job.

Job	When	Calls	SRS
Expire no-card holds	hold_expires_at passed	IReservation.releaseExpired	FR-RES-05, FR-RES-13
Expire waitlist offers	offer_expires_at passed	IWaitlist.expireOffer	FR-RES-06
Pickup window opens	pickup_start reached	INotification (bag ready)	FR-RES-11
Suggest a discount	deadline nearing with bags unreserved	IDiscount, INotification to the vendor	FR-LST-08
Automatic charity offer	TBD-5 minutes before pickup_end, vendor opted in, bags unreserved	IPartnerOffer.offer	FR-RTE-11
Window ended, nothing taken	pickup_end passed with no consumer or partner	INotification (ask the vendor to record landfill)	FR-RTE-13
Uncollected Zakat	pickup_end passed, donation not Delivered	IZakat.refundUncollected	E-TGI-03

6. Human Interface Design

6.1 Overview of the user interface

One app shows each role its own home screen after login (IF-UI-01). Every screen works in Arabic (right-to-left, mirrored) and English, and the language can be switched before registration (FR-ACC-09,

IF-UI-02, IF-UI-03). The sign-up, browse and reserve flows work with TalkBack and VoiceOver, and text meets a 4.5:1 contrast ratio (NFR-07). The screens follow SRS 3.1:

Role	Main screens
Any user	Language selection, registration, login, password reset, profile, impact numbers
Consumer	Nearby listings on a map and in a list with filters; listing details; reserve (card or no card); my reservations with the pickup code, check-in and cancel; waitlist; rating; donate as Zakat; AI assistant; Ramadan mode
Vendor	New listing (photo, AI draft, details, perishability, destination); Quick Post; my listings; reservations for a listing; enter pickup code; suggested discount; flag a no-show; automatic charity offers; landfill record; application status
Charity and composting partner	Application status; Donation or Compost offers; accept an offer and propose a pickup time; enter pickup code
Administrator	Pending applications; approve, reject, suspend or remove an account; flagged accounts

6.2 Screen images

Screen mock-ups will be added in a later sprint.

6.3 Screen objects and actions

- **Map and list of listings:** filter drawer (distance, price, pickup time, vendor rating); each listing shows its perishability category and pickup urgency, worded as how soon to collect, never as safe to eat (IF-UI-05).
- **Listing creation:** camera or gallery photo; AI draft fields marked as drafts until the vendor confirms each one; calorie range labelled "Estimate" with a remove button (IF-UI-04, NFR-05b).
- **Reserve:** card or no-card choice; the no-card option shows the hold end time and the "I'm on my way" button; a blocked consumer sees the block end date and the card option (E-RES-04).
- **My reservations:** the 6-digit pickup code, shown from the local cache when offline (IF-UI-07, E-RES-06).
- **Assistant:** text box and microphone button; every reservation is repeated back with Confirm and Cancel buttons (FR-AST-03).
- **Errors:** the invalid field is highlighted with a message in the user's language (IF-UI-06).

7. Requirements Matrix

All 55 SRS functional requirements trace to at least one component. C-01 Mobile App supports every requirement as the user's screens and C-27 Data Access stores every one, so they are listed only where they do more than that.

7.1 Functional requirements to components

FR	User story	Primary component	Supporting components
FR-ACC-01	US-01	C-03 Account	C-02
FR-ACC-02	US-02	C-03 Account	C-04
FR-ACC-03	US-03	C-03 Account	C-04
FR-ACC-04	US-04	C-02 API Layer	C-03
FR-ACC-05	US-05	C-03 Account	C-30
FR-ACC-06	US-06	C-03 Account	
FR-ACC-07	US-07	C-04 Admin & Approval	C-18
FR-ACC-08	US-08	C-04 Admin & Approval	C-02, C-18
FR-ACC-09	US-09	C-01 Mobile App	C-03
FR-LST-01	US-10	C-05 Listing	C-26, C-29
FR-LST-02	US-11	C-05 Listing	C-06
FR-LST-03	US-12	C-05 Listing	C-20
FR-LST-04	US-13	C-05 Listing	C-06
FR-LST-05	US-14	C-05 Listing	C-09, C-12, C-18
FR-LST-06	US-15	C-05 Listing	
FR-LST-07	US-16	C-05 Listing	C-20
FR-LST-08	US-17	C-08 Discount	C-19
FR-RTE-01	US-18	C-06 Perishability	
FR-RTE-02	US-18	C-06 Perishability	
FR-RTE-03	US-18	C-06 Perishability	C-05
FR-RTE-04	US-18	C-07 Routing & Feed	C-01
FR-RTE-05	US-19	C-07 Routing & Feed	C-27

FR	User story	Primary component	Supporting components
FR-RTE-06	US-19	C-07 Routing & Feed	
FR-RTE-07	US-19	C-07 Routing & Feed	C-13
FR-RTE-08	US-20	C-07 Routing & Feed	C-27, C-28
FR-RTE-09	US-21	C-07 Routing & Feed	C-27, C-28
FR-RTE-10	US-22	C-13 Partner Offer	C-11
FR-RTE-11	US-23	C-13 Partner Offer	C-19
FR-RTE-12	US-24	C-13 Partner Offer	C-11
FR-RTE-13	US-25	C-13 Partner Offer	C-16, C-19
FR-RES-01	US-26	C-07 Routing & Feed	C-24, C-28
FR-RES-02	US-27	C-09 Reservation	C-12, C-23
FR-RES-03	US-28	C-09 Reservation	C-19
FR-RES-04	US-28	C-09 Reservation	
FR-RES-05	US-28	C-09 Reservation	C-10, C-19
FR-RES-06	US-29	C-10 Waitlist	C-18, C-19
FR-RES-07	US-30	C-09 Reservation	C-27
FR-RES-08	US-31	C-09 Reservation	C-01
FR-RES-09	US-32	C-09 Reservation	
FR-RES-10	US-33	C-11 Pickup Code	C-13
FR-RES-11	US-34	C-18 Notification	C-19, C-25
FR-RES-12	US-35	C-09 Reservation	C-12
FR-RES-13	US-35	C-15 Rating & No-show	C-09, C-19
FR-AST-01	US-36	C-17 Assistant	C-07, C-21, C-22
FR-AST-02	US-37	C-17 Assistant	C-21, C-22
FR-AST-03	US-38	C-17 Assistant	C-09, C-12
FR-AST-04	US-39	C-17 Assistant	C-05, C-21
FR-TGI-01	US-40	C-15 Rating & No-show	

FR	User story	Primary component	Supporting components
FR-TGI-02	US-40	C-15 Rating & No-show	C-07
FR-TGI-03	US-41	C-15 Rating & No-show	
FR-TGI-04	US-42	C-15 Rating & No-show	C-04, C-18
FR-TGI-05	US-43	C-14 Zakat & Receipt	C-09, C-12, C-13
FR-TGI-06	US-43	C-14 Zakat & Receipt	C-19
FR-TGI-07	US-44	C-16 Impact	
FR-TGI-08	US-45	C-07 Routing & Feed	C-01

7.2 Nonfunctional requirements to components

NFR	Satisfied by
NFR-01 Search under 2 seconds	C-27, C-28 (PostGIS spatial index)
NFR-02 AI draft within 10 seconds	C-20 time limit, C-05 manual fallback
NFR-03 Works without AI	Fallbacks in C-05, C-06, C-17
NFR-04 Classifier accuracy	C-20 and the provider chosen under TBD-2
NFR-05 Food safety	C-05 (confirm every AI field, expiry cap), C-06 (Highly perishable default), C-01 (wording)
NFR-06, NFR-07 Arabic and accessibility	C-01
NFR-08 Slow connections	C-01 (compression, cache), C-02 (small payloads)
NFR-09 Security	C-02 (HTTPS, RBAC, lockout), C-03 (salted hashes)
NFR-10 Card data	C-12, C-23
NFR-11 Privacy	C-07 (location used, not stored), C-22 (audio deleted)
NFR-12 Availability	One adapter per external service with failure handling: C-23, C-24, C-25, C-30
NFR-13 Maintainability	Layers and adapters testable with mocks; GitHub Actions on every merge
NFR-14 Portability	C-01 React Native

8. Technical Risks and Appendices

8.1 Technical risks

ID	Risk	Impact	Mitigation
T-01	AI classification below 80% accuracy (NFR-04)	Wrong drafts, vendor frustration	Vendor confirms every field; test early on the 60-photo set; low confidence routes to manual entry and Highly perishable; only the six food categories in SRS Appendix A
T-02	AI uncertain or wrong about perishability	Pickup windows too long for the food	Rule-based estimate; Highly perishable by default; expiry caps the window; vendor may only choose a more perishable category; no safety certification
T-03	Two users reserve the last bag at once	Over-reservation (E-RES-02)	One transaction with a row lock on the listing; test with concurrent requests
T-04	Timers missed or repeated after a restart	Bags stuck or released twice	Expiry times stored in the database; the worker polls and updates idempotently; restart test
T-05	Payment sandbox limits (refunds, failures, Egyptian cards and wallets)	Payment and refund flows not fully testable	Choose a gateway with a sandbox early; mock gateway behind C-23; test refunds for cancelled listings and uncollected Zakat; the no-card flow works without the gateway
T-06	Arabic speech recognition weak on Egyptian dialect	Assistant misunderstands requests	Assistant is an extra way to use the app (Medium and Low priority); typed fallback; every reservation is repeated for confirmation; test with Egyptian Arabic samples
T-07	Slow connections (Fast 3G)	Slow uploads, failed calls (NFR-08)	Compress photos to 500 KB; small payloads; retries; cached reservations and pickup codes
T-08	External API cost or rate limits (AI, language model, speech, email and SMS)	Features stop during the demo	Adapters with time limits and fallbacks; budget caps; mock adapters for development

ID	Risk	Impact	Mitigation
T-09	Push notifications not delivered	Missed 10-minute waitlist offers	Offers also shown in the app; the offer lives in the database, not only in the push (IF-SW-06)
T-10	Location permission denied or inaccurate GPS	Wrong or no nearby results	Explain why location is needed (IF-HW-04); manual location entry depends on TBD-3; location used only for the request, never stored (NFR-11a)
T-11	Role-based access mistakes	A user reaches another role's actions	RBAC in C-02 only; an automated 403 test for every pair of roles (NFR-09c)
T-12	Voice recordings kept by the speech provider	Privacy breach (NFR-11b)	Choose a provider with no retention; delete audio after processing
T-13	Android and iOS differences (RTL, TalkBack, VoiceOver)	Accessibility failures (NFR-06, NFR-07)	Test on both platforms from the first sprint; accessibility scanner
T-14	Backend process crashes or hosting downtime	App cannot reach the API	Automatic restart policy and health check on both processes; cached reservations stay visible

8.2 Appendices

- **Appendix A, Glossary:** see SRS Appendix A; the terms specific to this document are in Section 1.5.
- **Appendix B, Open design items:** the open values in SRS Appendix C (TBD-1 to TBD-5) are settings in this design, not changes to the architecture: TBD-1 sets the minimum app versions, TBD-2 picks the backend framework and the provider behind each adapter, TBD-3 decides manual location entry in C-01, TBD-4 sets the password rules in C-03, and TBD-5 sets the automatic charity offer time in the settings table used by C-19.